

科目：データベース演習

第13回：SQL応用：アクセスログのセッション分析 (入門)

講師：鈴木源吾

前回の復習

1. サブクエリ: 問合せの結果を問合せで利用する
2. ウィンドウ関数でグループ全体を対象にした計算をする

今回のトピック

1. セッション分析：アクセス間隔を求める
 - LAGウィンドウ関数の利用
2. セッション分析：セッション識別
 - CASE式の利用

その他の資料：第10章

topic 1

セッション分析：アクセス間隔を求める

今回の内容について

- 今回の内容は、その他の資料「10年戦えるデータ分析入門」の第10章の、アクセスログのセッション分析の、最初の部分を解説する
- やや中途半端で終わるため、興味のある人は、続きを読んで見ることに

セッション分析とは

- セッションとは、ユーザーがウェブサイトを利用しているひとまとまりの時間、あるいは、その時間に発生するウェブサイトへのアクセス
- セッションの例
 - 商品を見る→他の商品も見比べて比較する→購入する
 - （Googleの検索結果から飛んできて）何かのページを見て終わり
- セッション分析：セッションの特徴を分析すること
 - セッションが長い・短い
 - どこからこのサイトへ来たか
 - このサイト内でどんな行動をとったか

セッション分析の例

その他の資料「10年戦えるデータ分析入門」では、

顧客が商品検索をした後にカートに商品を入れる率

を求めている。これはコンバージョン率と呼ばれる

- サイトにおいて何かを達成することを「コンバージョン（顧客転換）」と呼ぶため（マーケティング用語）。この場合は、カートに商品を入れることを達成

これを求めるために、「パスパターンの頻度分析」を行っている

- パスパターンとは、Webサイトにおけるユーザーの移動のパターン
- 頻度分析なので、頻繁に起こるパスパターンを調べる分析

セッションナイズ

しかし、今回は、セッション分析の入り口の**セッションナイズ**のみ解説する

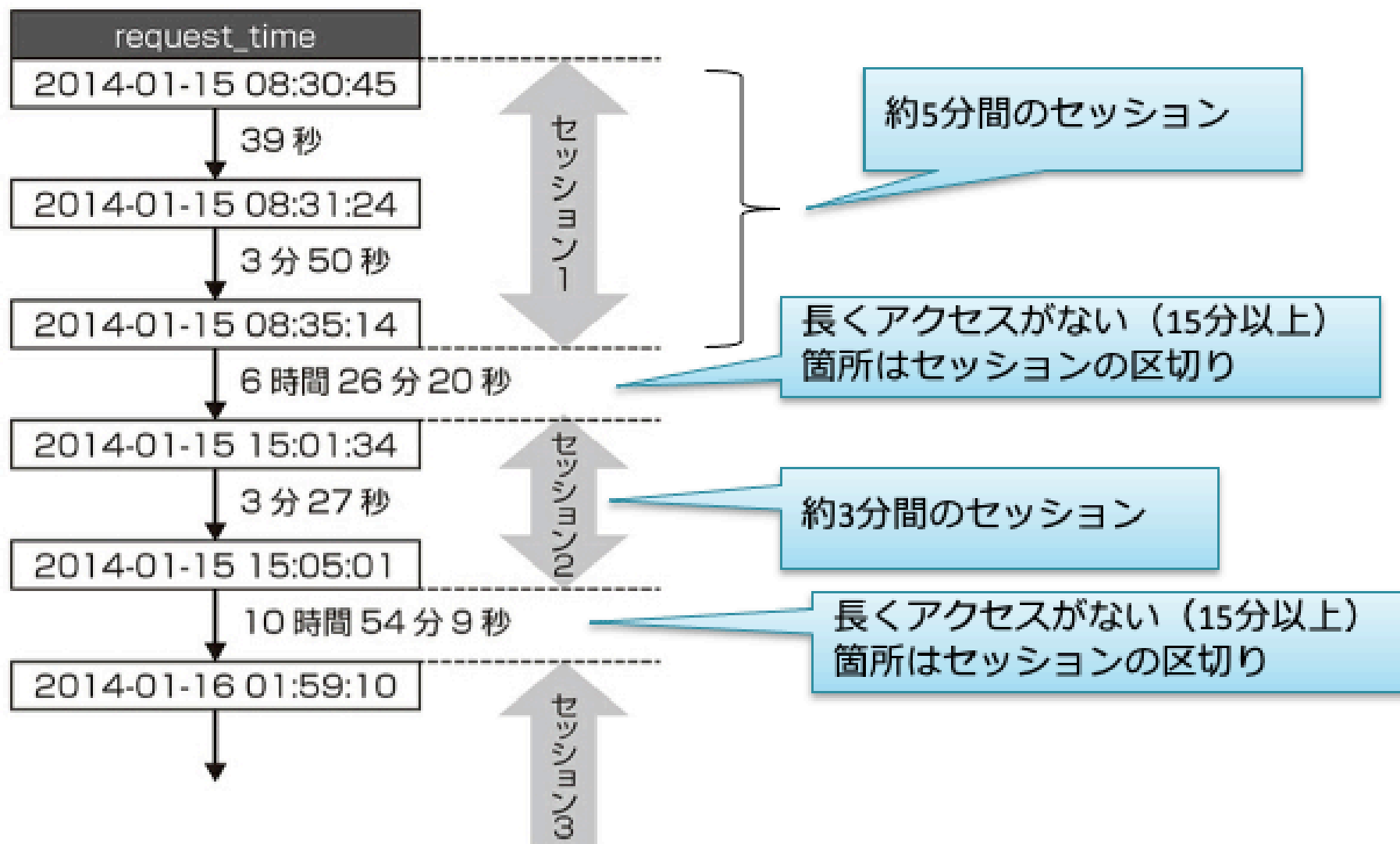
セッションナイズ：アクセスが所属するセッションを決めること

- ユーザーは様々なアクセスを行う
- ある1人のユーザーの「一連の」アクセスがセッション

→ 今回は、**同一のユーザーの連続するアクセスで、アクセス間隔が15分未満**である一連のアクセスを1つのセッションとする

- アクセス間隔とは、時系列で隣接するアクセスのアクセス時刻の差とする
- このやり方は、時間で単純に判断するシンプルな方式
- 他にもセッションナイズの方式はいくつかある

アクセス間隔によるセッションナイズ



アクセス間隔を求めよう

access_logテーブルのrequest_timeカラム（アクセスのあった時刻）を利用する
まずは、「連続する」2つの行のrequest_timeの差を求めたい

- これは、その2つのアクセス時刻の間隔を表す（同一ユーザとは限らない）
- この差を求めるには、一つ前（または一つ後）の行をとる機能がほしい

	request_time timestamp without time zone	
20	2014-07-11 21:41:13	} 15秒
21	2014-07-11 21:41:28	
22	2014-07-11 21:43:19	} 1分51秒
23	2014-07-11 21:45:28	
24	2014-08-03 23:11:39	} 1ヶ月以上
25	2014-09-21 02:56:00	

連続する2つの行の
request_timeの差

1つ前の行の値をとる：LAGウィンドウ関数

LAGウィンドウ関数は、1つ前の行の値を取得できる
まずは、以下のシンプルな例で確認してみよう

```
1 SELECT request_time, LAG(request_time) OVER ()  
2 FROM access_log LIMIT 100;
```

OVERのカッコの中は空.
PARTITIONはテーブル全体.

データ出力 実行計画 メッセージ 通知

	request_time timestamp without time zone	lag timestamp without time zone
1	2012-01-01 00:00:02	[null]
2	2012-01-01 00:02:45	2012-01-01 00:00:02
3	2012-01-01 00:04:07	2012-01-01 00:02:45
4	2012-01-01 00:01:43	2012-01-01 00:04:07
5	2012-01-01 00:03:20	2012-01-01 00:01:43

確かに前の行の値を取得

LAGウィンドウ関数を使ってアクセス間隔を求める

前ページから，連続する2つの行のrequest_timeの差は，以下で求められる

```
request_time - LAG(request_time)
```

さらに，同一ユーザの連続するアクセスでこれを利用するには，

- セッションは，異なるユーザが混じってはいけなから，アクセス間隔も，同一のユーザのアクセス内で決める必要がある
- アクセスログは，request_timeカラムの値の順番に並んでいるとは限らない（RDBMSは行の順序は保証されない）．よって，ソートが必要である

アクセス間隔を求めるSQL文

```
1 SELECT customer_id, request_time, request_time - LAG(request_time) OVER (  
2     PARTITION BY customer_id  
3     ORDER BY request_time  
4 ) AS access_interval  
5 FROM access_log LIMIT 100;
```

customer_id毎にパーティション（グループ）を作る
= 同じcustomer_idの中で前の行を見る

request_timeでソートする

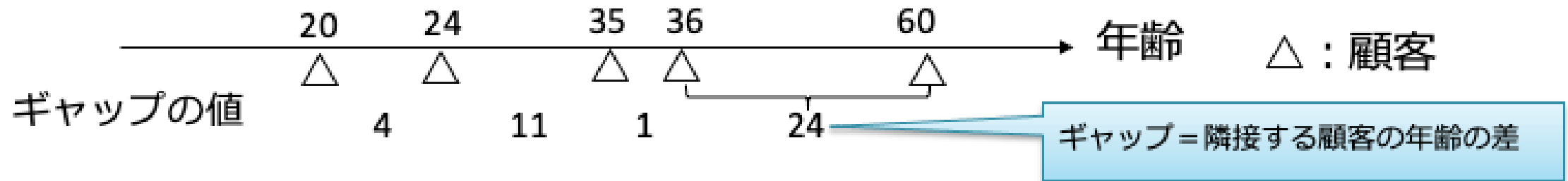
データ出力 実行計画 メッセージ 通知

	customer_id integer	request_time timestamp without time zone	access_interval interval
1	1	2012-03-04 18:17:07	[null]
2	1	2012-05-15 20:19:25	72 days 02:02:18
3	1	2012-05-24 10:33:50	8 days 14:14:25
4	1	2012-08-28 07:45:21	95 days 21:11:31
5	1	2012-11-03 18:48:18	67 days 11:02:57

アクセス間隔

例題1

customersテーブルは顧客の情報である，顧客のID（customer_idカラム），年齢（customer_ageカラム）などを管理している．顧客の年齢にギャップがどの程度あるかを調べたい．年齢のギャップとは，年齢が小さい順に顧客を並べたときの隣接する2人の顧客の年齢の差を意味することとする．LAGウィンドウ関数を用いることにより，顧客のID，顧客の年齢，年齢のギャップ（AS句を用いage_gapという名前にせよ）の組を，age_gapの大きい順に並べ，最大10行のみ求めるSQL文を作成せよ



例題1のSQL文の実行例

- age_gapがNULLである行（customer_ageが最も小さい顧客に関する行）が先頭に来るが、この行は排除しなくとも良いこととする。以下の結果から年齢のギャップは最大1であることがわかる。

	 customer_id [PK] integer 	customer_age integer 	age_gap integer 
1	51874	10	[null]
2	77304	18	1
3	5040	12	1
4	23262	14	1
5	92790	16	1
6	95546	17	1
7	98412	15	1
8	98754	11	1
9	94968	13	1

演習 課題6-3

itemsテーブルは商品の情報である、商品のID (item_idカラム)、名前 (item_nameカラム) 価格 (item_priceカラム) などを管理している。商品の価格にギャップがどの程度あるかを調べたい。価格のギャップとは、価格が安い順 (安い商品→高い商品の順) に商品を並べたときの隣接する2つの商品の価格の差を意味することとする。LAGウィンドウ関数を用いることにより、商品のID、商品の名前、商品の価格、価格のギャップ (AS句を用いprice_gapという名前にせよ) の組を、price_gapの大きい順に並べ、最大10行のみ求めるSQL文を作成せよ

演習 課題6-3のSQL文の実行例

- price_gapがNULLである行（item_priceが最も小さい商品に関する行）が先頭に来るが、この行は排除しなくとも良いこととする。以下の結果から価格のギャップは最大7700であることがわかる。

	item_id [PK] integer	item_name text	item_price integer	price_gap integer
1	49	First Tシャツ	1210	[null]
2	88	Laser ジャケット	28580	7700
3	3	Rose ジャケット	13010	2240
4	85	Laser バッグ	17330	910
5	211	Laser ジャケット	29480	900
6	56	Extra ジャケット	19260	830
7	135	Z3 ジャケット	14970	690
8	66	First カットソー	1820	520
9	384	Extra ジャケット	20880	450

topic 2

セッション分析：セッション識別

セッション識別

セッションの定義は,

「同一のユーザーの連続するアクセスで、アクセス間隔が15分未満である一連のアクセス」

だった. topic 1で、アクセス間隔を求めることができたから、さらにセッションを識別する（決める）ためには、アクセス間隔が15分未満であるような、アクセス（の組）を見つける必要がある

セッション識別で欲しい結果

アクセスログに対して、アクセス間隔を求め、以下のようにセッションを表す番号を付与できればよい

request_time	アクセス間隔	セッション番号
2014-01-15 08:30:45	(なし)	1
2014-01-15 08:31:24	39 秒	1
2014-01-15 08:35:14	3 分 50 秒	1
2014-01-15 15:01:34	6 時間 26 分 20 秒	2
2014-01-15 15:05:01	3 分 27 秒	2
2014-01-16 01:54:10	10 時間 54 分 9 秒	3

15分以上アクセス間隔があいたら新しいセッションの開始

15分以上のアクセス間隔

セッション番号の求め方

以下の2つの組合せで、セッション番号は求められる

- セッションの先頭→1, 先頭でない→0というフラグを求める
- セッション先頭フラグの累積和が、セッション番号

セッションの先頭を
表すフラグを求める

request_time	アクセス間隔	セッション先頭フラグ	セッション番号
2014-01-15 08:30:45	(なし)	1	1
2014-01-15 08:31:24	39 秒	0	1
2014-01-15 08:35:14	3 分 50 秒	0	1
2014-01-15 15:01:34	6 時間 26 分 20 秒	1	2
2014-01-15 15:05:01	3 分 27 秒	0	2
2014-01-16 01:59:10	10 時間 54 分 9 秒	1	3

セッション先
頭フラグの累
積和がセッ
ション番号に
なる！

セッション番号の求め方

セッション先頭フラグの求め方

- アクセス間隔が15分以上→1
- アクセス間隔が15分未満→0

とすればよい → CASE式を用いる（後で説明する）

セッション番号の求め方

- セッション先頭フラグの累積和 → SUMウィンドウ関数を使う。
 - OVER句で「ROWS BETWEEN 前側の行 AND 後側の行」と指定できる機能を利用し、現在の行の前のすべてを合計する
 - 今回は説明しない。興味のある人は、「10年使えるデータ分析」を参照
 - 本トピックの最後に、参考としてセッション番号を求めるSQL文を掲載する

SQLにおけるCASE式

CASE式とは、カラムの値の一致や、条件式の結果により、返却する値を変えるときに使う式である。返却値は新しく定義したカラムの値として利用される

→ プログラミング言語における条件分岐と同等

CASE式に、単純CASE式と検索CASE式がある

- 単純CASE式：カラムの値により分岐し、返却する値を変える
- 検索CASE式：条件式の結果により分岐し、返却する値を変える

単純CASE式

単純CASE式の文法は一般には以下となる

- 条件の対象となるカラムの値が、 WHEN句のカラムの値に一致するかを上から調べる
- 最初に一致したときに、 そのTHEN句にある返却値が返される
- すべての条件に一致しない場合、 ELSE句にある返却値が返される

```
CASE 条件の対象となるカラム名  
  WHEN カラムの値1 THEN 返却値1  
  WHEN カラムの値2 THEN 返却値2  
  WHEN カラムの値3 THEN 返却値3  
  ...  
  ELSE 返却値n  
END
```

単純CASE式の例

例えば、customersテーブルで、customer_name カラムの値とあわせて、customer_genderカラムの値がFだったら女性、Mだったら男性、それ以外ならその他の値を返すSQL文は、以下となる

```
SELECT customer_name,  
       CASE customer_gender  
         WHEN 'F' THEN '女性'  
         WHEN 'M' THEN '男性'  
         ELSE 'その他'  
       END  
FROM customers;
```

検索CASE式

検索CASE式の文法は一般には以下となる

- WHEN句の条件式が真になるかを上から調べる
- 最初に真になったときに、そのTHEN句にある返却値が返される

```
CASE  
  WHEN 条件式1 THEN 返却値1  
  WHEN 条件式2 THEN 返却値2  
  WHEN 条件式3 THEN 返却値3  
  ...  
  ELSE 返却値n  
END
```

検索CASE式の利用例

例えば、itemsテーブルを用いて、商品に関して、item_nameカラムの値と合わせて、価格（item_priceカラム）が1万円より高かったら「高い!」、価格が3000円より安かったら「安い!」、そうでなかったら「そこそこ」を、カラムの別名に「評価」とつけて返すSQL文は以下となる

```
SELECT item_name, item_price,  
       CASE  
           WHEN item_price > 10000 THEN '高い!'  
           WHEN item_price < 3000 THEN '安い!'  
           ELSE 'そこそこ'  
       END AS 評価  
FROM items;
```

セッション先頭フラグを求めるSQL文

- 最初の行だけは、アクセス間隔を求めるとnullになる（一つ前のタプルがないから）。この行は最初のセッションの先頭なので、先頭フラグを1とする
- その後は、アクセス間隔が15分以上であれば1として、それ以外を0とすればよい
→CASE式が利用できる！

request_time	アクセス間隔	セッション先頭フラグ	セッション番号
2014-01-15 08:30:45	(なし)	1	1
2014-01-15 08:31:24	39 秒	0	1
2014-01-15 08:35:14	3 分 50 秒	0	1
2014-01-15 15:01:34	6 時間 26 分 20 秒	1	2
2014-01-15 15:05:01	3 分 27 秒	0	2
2014-01-16 01:59:10	10 時間 54 分 9 秒	1	3

この先頭だけ例外

アクセス間隔が15分以上

セッション先頭フラグを求めるSQL文

別名access_intervalを、SELECT句内で参照できないため冗長な記述要
時刻の差はinterval型となるため、15分を"interval '15 minute'"と記述する必要あり

```
SELECT customer_id, request_time, request_time - LAG(request_time) OVER (  
    PARTITION BY customer_id  
    ORDER BY request_time  
    ) AS access_interval,  
    CASE  
        WHEN request_time - LAG(request_time) OVER (  
            PARTITION BY customer_id  
            ORDER BY request_time  
            ) is null THEN 1  
        WHEN request_time - LAG(request_time) OVER (  
            PARTITION BY customer_id  
            ORDER BY request_time  
            ) >= interval '15 minute' THEN 1  
        ELSE 0  
    END AS session_start_flag  
FROM access_log LIMIT 100;
```

参考：セッション番号を求めるSQL文

```
SELECT customer_id, request_time, session_start_flag, SUM(session_start_flag) OVER (
    PARTITION BY customer_id
    ORDER BY request_time
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS session_id
FROM (
    SELECT customer_id, request_time, request_time - LAG(request_time) OVER (
        PARTITION BY customer_id
        ORDER BY request_time
    ) AS access_interval,
    CASE
        WHEN request_time - LAG(request_time) OVER (
            PARTITION BY customer_id
            ORDER BY request_time
        ) is null THEN 1
        WHEN request_time - LAG(request_time) OVER (
            PARTITION BY customer_id
            ORDER BY request_time
        ) >= interval '15 minute' THEN 1
        ELSE 0
    END AS session_start_flag
    FROM access_log
);
```

参考：セッション番号を求めるSQL文の実行結果

間隔が15分未満のアクセスは同一セッションとなっている（結果は一部のみ）

	customer_id integer	request_time timestamp without time zone	session_start_flag integer	session_id bigint
1	1	2012-03-04 18:17:07	1	1
2	1	2012-05-15 20:19:25	1	2
3	1	2012-05-24 10:33:50	1	3
4	1	2012-08-28 07:45:21	1	4
5	1	2012-11-03 18:48:18	1	5
6	1	2012-11-18 21:26:20	1	6
7	1	2012-12-24 19:06:21	1	7
8	1	2013-03-18 00:25:20	1	8
9	1	2013-04-08 09:10:39	1	9
10	1	2013-05-08 11:12:29	1	10
11	1	2013-05-08 11:13:24	0	10
12	1	2013-05-08 11:13:55	0	10
13	1	2013-05-08 11:15:40	0	10
14	1	2013-05-08 11:16:12	0	10
15	1	2013-08-15 16:06:59	1	11

例題2

顧客を表すcustomersテーブルを用いて、氏名（customer_nameカラム）と、年齢（customer_ageカラム）その人が属する年齢グループ（別名をage_groupとせよ）の組を、最大10行、返却するSQL文を作成せよ。年齢グループの値は、30歳未満：若者、30歳以上50歳未満：中年、50歳以上：老人、という値をとることとする。検索結果例を以下に示す

	customer_name character varying (32)	customer_age integer	age_group text
1	白井 奈月	37	中年
2	三谷 朝陽	16	若者
3	早川 彩	39	中年
4	谷口 優一	46	中年
5	阪本 美帆	24	若者
6	瀬戸 美菜	28	若者
7	若山 みゆき	16	若者
8	稲葉 薫	26	若者
9	高柳 優	15	若者
10	大西 朝香	50	老人

演習 課題6-4

顧客を表すcustomersテーブルを用いて、顧客のID（customer_idカラム）、氏名（customer_nameカラム）と、居住地（customer_locationカラム）その人が新潟県に住んでいるかどうかのフラグ（別名をniigata_flagとせよ）の組を、最大10行、返却するSQL文を作成せよ。niigata_flagの値は、居住地が新潟県である場合に1をとり、それ以外の都道府県である場合は0をとることとする。検索結果例を以下に示す

	customer_id [PK] integer	customer_name character varying (32)	customer_location text	niigata_flag integer
1	1	白井 奈月	新潟県	1
2	2	三谷 朝陽	京都府	0
3	3	早川 彩	東京都	0
4	4	谷口 優一	神奈川県	0
5	6	阪本 美帆	千葉県	0
6	7	瀬戸 美菜	福岡県	0
7	8	若山 みゆき	愛媛県	0
8	9	稲葉 薫	福島県	0
9	10	高柳 優	埼玉県	0
10	11	大西 朝香	香川県	0